

Phizura: On-the-Fly Music Recording with Method Proxies

Domenico Cipriani¹, Sebastian Jordan Montaña² and Stéphane Ducasse²

¹Evref, Inria, CNRS, Centrale Lille, UMR 9189 CRISTAL, Park Plaza, Parc scientifique de la Haute-Borne, 40 Av. Halley Bât A, 59650 Villeneuve-d'Ascq, France

²Univ. Lille, Inria, CNRS, Centrale Lille, UMR 9189 CRISTAL, Park Plaza, Parc scientifique de la Haute-Borne, 40 Av. Halley Bât A, 59650 Villeneuve-d'Ascq, France

Abstract

Live coding is a performance practice in which artists write and modify code in real time to generate music. A key challenge is capturing a performance for later replay, sharing, or analysis without resorting to audio recording. This paper presents Phizura, a recording tool for on-the-fly music performances conducted with Coypu, a Pharo-based library and dialect for live music programming. Phizura leverages Method Proxies, a safe and fast message-passing control library, to non-intrusively intercept Playground evaluations and keystrokes during a performance. The recorded data, timestamped expressions and input events, can later be emitted as executable Pharo code that faithfully replays the performance with the original timing. We describe the architecture of Phizura, detail how Method Proxies enable a clean separation between recording logic and the Coypu codebase, and compare the approach to a previous Announcer-based implementation.

Keywords

Pharo, Method Proxies, Coypu, On-the-fly music programming

1. Introduction

Live coding is an increasingly popular creative practice in which performers write and modify source code in real time to produce music, visuals, or other time-based media¹. The field has developed a substantial academic presence: the International Conference on Live Coding (ICLC), now in its tenth year, has served as the primary venue for peer-reviewed research on the practice, attracting contributions from computer science, music, and the performing arts. Programming languages such as Tidal Cycles[?], Sonic Pi[?], and Orca[?] have popularized the practice, each offering alternative syntaxes and notations for expressing musical patterns. Within the Pharo ecosystem, Coypu[?] provides a library and a DSL for on-the-fly music programming, enabling performers to construct rhythmic sequences, melodic patterns, and to modify them interactively in a Playground.

To date, relatively little scholarly attention has been devoted to capturing and preserving live coding performances, particularly the generative process itself. Audio recording captures the sonic result but discards the compositional process: the sequence of code edits, evaluations, and their precise timing. Audio files are also large, difficult to share, and impossible to re-execute under a different setup. Compositional data, by contrast, enables exact replay of a performance with alternative sound synthesis, facilitates sharing between musicians as lightweight text files, supports analysis of live coding practice, and opens the door to automated post-processing of performances. This concern has been raised explicitly in the Tidal Cycles community², where users have discussed the need for tools that store every evaluation with its timestamp and allow session-based replay. However, no integrated solution has emerged so far.

This paper presents the design and implementation of Phizura, shows how Method Proxies[?] can be applied beyond profiling and debugging to support creative coding workflows, and compares the current approach to the previous Announcer-based implementation. The paper begins with background

IWST 2026: International Workshop on Smalltalk Technologies, July 4–10, 2026, Plovdiv, Bulgaria

✉ mspgate@gmail.com (D. Cipriani); sebastian.jordan@inria.fr (S. Jordan Montaña); stephane.ducasse@inria.fr (S. Ducasse)

ORCID 0009-0005-3023-3192 (D. Cipriani); 0000-0002-7726-8377 (S. Jordan Montaña); 0000-0001-6070-6599 (S. Ducasse)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

¹TOPLAP Manifesto

²Tidal Club Forum, How to replay a live coding session, link, 2020.

on Coypu and Method Proxies, then discusses the limitations of the earlier Announcer-based approach that motivated the redesign. It continues with the architecture and implementation of Phizura, the code emission and replay mechanism, and a discussion of related work. It closes with a reflection on generality, limitations, and future directions.

2. Background

2.1. Coypu: programming music on-the-fly with Pharo

Coypu[?] is a library and domain-specific dialect for live music programming within the Pharo environment. It acts as a client for an audio server, which can be either internal (a Phausto[?] DSP running within Pharo) or an external OSC-capable³ application, typically SuperCollider via the SuperDirt engine, or a MIDI device. A performer works in a Pharo Playground, writing expressions that define rhythmic and melodic patterns and assign them to named instruments. Once evaluated, these patterns begin playing immediately. From that point on, the performer reshapes the music in real time: modifying patterns, muting or soloing individual voices, swapping instruments, and layering new material over what is already running. Every evaluation hits the running output immediately. The performance is shaped by what gets evaluated, when, and in what order and that is exactly what Phizura records.

2.2. Method Proxies: dynamic method interception in Pharo

Method Proxies[?] is a message-passing control library for Pharo that lets you dynamically instrument any method to run user-defined actions before and after its execution. The idea is simple: you tell the framework which method you want to intercept, and you hand it a handler that says what to do around it. This handler⁴ defines two hooks: `beforeExecutionWithReceiver:arguments:`, which fires right before the original method runs, and `afterExecutionWithReceiver:arguments:returnValue:`, which fires after it returns. That is all the user needs to care about — everything else is managed under the hood. Internally, when a method is instrumented, the framework replaces the original compiled method in the method dictionary with a copy of a pre-compiled trap method template. Through literal patching, this trap wires in references to the actual handler and to the original method, which is preserved under a mangled selector. The trap leverages Pharo's stack unwinding mechanism via a marker primitive to guarantee execution of the after-hook without allocating block closures, making it significantly faster than `ensure:-`-based approaches. Method Proxies also takes care of the tricky parts. A per-thread meta-level detection mechanism prevents infinite recursion when the instrumentation code itself triggers instrumented methods. Thread safety, non-local returns, exceptions, and clean instrumentation are all handled by the framework. The entry point is `MpMethodProxy`, which manages installing and removing the instrumentation. The users just define what happens before and after, and the result is code that is easier to maintain and extend.

2.3. Previous approaches to capturing performances

The first recording mechanism for Coypu was implemented by Redwane Engels during an internship and demonstrated at ESUG 2024 in Lille. It relied on Pharo's `Announcer` and `Announcement` classes, which implement the observer design pattern.

The approach was straightforward: the Coypu codebase itself was modified so that at each point where an evaluation could produce a musical effect, an announcement was explicitly fired. A subscriber object, the recorder, registered interest in these announcements and logged each one with a timestamp.

It functioned as intended, but introduced significant trade-offs. Recording logic was distributed throughout the codebase, and supporting each new recordable event required direct modifications to

³Open Sound Control (OSC) is a protocol for real-time communication between multimedia devices and software, typically transmitted over UDP, <https://opensoundcontrol.stanford.edu>.

⁴An instance of a subclass of `MpHandler`.

the source code. The recorder depended on specific announcement classes defined within the library, making the two systems difficult to evolve independently. Extending it to capture additional events, such as keystrokes, cursor positions, or events from other tools, required yet more invasive changes. And every modification to the evaluation pipeline risked breaking or silently bypassing the recording hooks. These limitations motivated the redesign using Method Proxies, which externalizes the instrumentation entirely.

3. Design and Implementation

Phizura is organized in three packages: Phizura (core recording and emission logic), Phizura-Playground (UI integration), and Phizura-Tests (test suite). The architecture addresses four concerns: instrumentation, recording, code emission, and UI integration.

3.1. Instrumentation

At the core of Phizura’s design is a small class hierarchy for instrumentation. `PhizuraInstrumentor` is an abstract class that manages a collection of `MpMethodProxy` instances and a shared `PhizuraHandler`. Subclasses define which methods to instrument by implementing `initializeMethodProxies`.

```
PhizuraInstrumentor >> initialize
  super initialize.
  instrumentationHandler := PhizuraHandler
    onRecorder: recorder.
  self initializeProxyVariables.
  self initializeMethodProxies
```

Listing 1: Initialization of `PhizuraInstrumentor`

The concrete subclass `PhizuraEvaluationInstrumentor` instruments two methods: `SpCodeSelectionCommand»#evaluate:andDo:`, the standard Pharo Playground evaluation command, and `PhizuraPlaygroundPagePresenter»#doEvaluateAndGoFor:`, the Phizura Playground’s own evaluation path. Both share the same handler instance, so all evaluations are recorded uniformly regardless of which path triggers them.

The handler itself, `PhizuraHandler`, is a subclass of `MpHandler` and constitutes the only user-defined hook in the entire system. It implements a single method:

```
PhizuraHandler >> beforeExecutionWithReceiver:
  anObject arguments: anArrayOfArguments
  | argument |
  argument := anArrayOfArguments first.
  recorder addRecord:
    (PhizuraEvaluationRecordEntry new
      time: DateAndTime now;
      expression: argument;
      cursorPosition:
        playgroundPage text cursorPosition;
      yourself)
```

Listing 2: `PhizuraHandler` before-execution hook

The handler extracts the first argument of the intercepted method, which is the expression string being evaluated, and creates a record entry with three pieces of information: the current timestamp, the expression text, and the cursor position in the editor. No Coypu method is modified, subclassed, or even referenced. The instrumentation is purely additive.

3.2. Recording

Phizura captures two kinds of events, modeled by a small class hierarchy rooted at `PhizuraBasicRecordEntry`. `PhizuraEvaluationRecordEntry` stores a code expression, a timestamp, and the cursor position, corresponding to Playground evaluations. `PhizuraKeyEventRecordEntry` stores a keyboard key, a timestamp, and the cursor position, captured via a `whenKeyUpDo:` event handler on the Playground's text presenter, independently of Method Proxies. Both entry types participate in double dispatch for code emission (see Section 5).

`PhizuraRecorder` ties the system together. It holds a collection of record entries, a reference to the instrumentor, and a boolean recording flag. When `record` is called, it creates a `PhizuraEvaluationInstrumentor`, installs the proxies, and sets the flag. When `stopRecording` is called, it uninstalls all proxies. A guard `isRecording` `ifTrue:` ensures that evaluations outside a recording session are not captured. For emergency recovery, a class-side `forceUnrecord` method uninstalls all proxies in the image.

3.3. The Phizura Playground

Phizura requires a custom Playground to intercept the text that is evaluated when the user presses `Cmd+D` (Do it). The standard Pharo Playground does not expose the evaluated expression in a way that can be captured externally. The expression text flows through internal methods that are not designed for observation. To solve this, Phizura provides its own Playground, built by subclassing the standard Pharo Playground classes at four points.

`PhizuraPlaygroundPresenter`, a subclass of `StPlaygroundPresenter`, manages the window lifecycle. It ensures that recording stops when the window is closed and creates instances of `PhizuraPlaygroundPage` instead of the default page.

`PhizuraPlaygroundPagePresenter`, a subclass of `StPlaygroundPagePresenter`, is the central customization point. It defines the method `doEvaluateAndGoFor:`, which receives the selected text as its argument and evaluates it. This method is one of the two targets instrumented by Method Proxies, and it is where the handler intercepts the expression being evaluated. In addition, this presenter initializes the recorder, registers a `whenKeyUpDo:` event handler to capture keystrokes, sets up a custom interaction model via `PhizuraPlaygroundInteractionModel`, and extends the toolbar with five commands: *Phizurear* (toggle recording), *Clean Phizura* (reset the recorder), *Inspect Phizura* (open an inspector on the recorder), *Open MoofLOD* (emit code with OSC messages), and *Open Records* (emit clean performance code).

`PhizuraPlaygroundPage`, a subclass of `StPlaygroundPage`, overrides `defaultObjectInspectorClass` to return `PhizuraPlaygroundPagePresenter`, ensuring the custom presenter is used. `PhizuraPlaygroundInteractionModel`, a subclass of `StPlaygroundInteractionModel`, allows binding a custom `doItReceiver`, giving the Playground page presenter access to itself as the evaluation context.

A global variable `PhiRecorderInstance` holds the active recorder, initialized when the Phizura Playground opens. The Phizura Playground is accessible from the Pharo world menu and can be opened programmatically via `PhizuraPlayground open`.

4. Code Emission and Replay

A key feature of Phizura is its ability to emit executable code from recorded data. The `PhizuraCodeEmitter` class iterates over the record entries and produces Pharo source code that, when evaluated inside a forked block, replays the performance with the original timing. The emitter computes the delay between consecutive entries by subtracting their timestamps. Each delay is then expressed as a `wait` statement in the generated code, so that when the emitted script runs in a forked process, the original temporal structure of the performance is faithfully reconstructed.

4.1. Emission Modes

Phizura supports two code emission modes, implemented via double dispatch between the emitter and the record entries.

The first mode, accessed through `emitPerformanceCode`, produces clean, executable Coypu code. Keystroke events are filtered out and only evaluation records are emitted, separated by the appropriate delays. The resulting script is self-contained: anyone with the same Coypu setup and sound definitions can evaluate it to hear the exact performance.

```
[
ep := EcoPhausto new.
1.0 seconds wait.
ep playFor: 128 bars.
1.5 seconds wait.
#(1 0 0 1 0 0 0 0) asSeq to: #kick.
1.8 seconds wait.
#rumba asRhythm to: #conga.
] fork.
```

Listing 3: Example of emitted performance code

The second mode, accessed through `emitMoofLODCode`, extends the performance code with OSC-compatible messages. After each expression, a string representation is sent via `toLocal: 'evaluatedCommand'`. Keystroke events emit the key value and cursor coordinates. This mode enables external visualization tools or network-based synchronization during replay.

Both modes rely on double dispatch on the record entry side, through the methods `emitPerformanceCodeOnEmitter:onStream:` and `emitMoofLODCodeOnEmitter:onStream:`. This makes it straightforward to add new emission formats without modifying the record entry classes or the emitter's core iteration logic.

5. Related Work

The challenge of recording and replaying live coding sessions has received relatively little attention in the literature. In the Tidal Cycles community, a forum discussion⁵ explicitly outlines requirements that Phizura addresses: storing every evaluation with its timestamp, handling sessions with start and stop, and enabling replay. The thread concludes without a solution, noting the absence of such a tool across live coding environments.

The most directly comparable work is Live Writing by Lee and Essl[?], presented at ICLC 2015. During a Gibber live coding session in the browser, Live Writing records keystroke-level data: every character typed, cursor position, and timestamp. The system captures the typing performance, that is, how the code was physically entered. Live Writing is implemented via CodeMirror editor hooks in the browser, whereas Phizura uses reflective instrumentation through Method Proxies, requiring no modification to the editor or the live coding library.

Another related effort is SHARP[?], a Pulsar⁶ extension that visualizes the history of each instrument a live coder creates, providing an interactive tree-like structure to track how blocks of code evolve over time and to revisit previous states. It addresses the versioning aspect of the creative process. SHARP manages the evaluated expressions but does not capture timing or produce replayable scripts. Phizura and SHARP are complementary: SHARP preserves the structural history of code, while Phizura preserves the temporal performance.

⁵Tidal Club Forum, How to replay a live coding session, link, 2020.

⁶Pulsar (<https://pulsar-edit.dev>) is a community-maintained, open-source text editor built on Electron, forked from GitHub's Atom after its discontinuation.

6. Hindsight

6.1. Comparison with the Announcer-based approach

Table 1 summarizes the key differences between the two recording implementations. The most significant advantage of the Method Proxies approach is the complete elimination of coupling between the recording infrastructure and the domain library. In the Announcer-based design, every new recordable event required a two-step modification: first, defining a new Announcement subclass in Coypu, and second, inserting the corresponding announcement firing at the right point in the evaluation pipeline. Over time, this scattered recording logic throughout the codebase, making it increasingly difficult to distinguish Coypu’s core musical functionality from the recording mechanism layered on top of it.

Table 1

Comparison of the Announcer-based and Method Proxies-based recording approaches.

Aspect	Announcer-based	Method Proxies-based
Coypu modifications	Required at each recordable site	None
Coupling	Tight (shared announcement types)	None (external instrumentation)
Adding new events	Modify Coypu + define announcement	Intercept target method
Removal	Must revert Coypu changes	Uninstall instrumentation
Keystroke capture	Not supported	Supported
Code emission	Not supported	Two modes (Performance, MoofLOD)

With Method Proxies, the instrumentation is installed externally on existing methods, and the handler receives the method’s arguments reflectively. The original methods are entirely unaware of the instrumentation. Recording a new kind of event only requires intercepting the relevant method. No Coypu code needs to change. Removing the recording capability is equally straightforward: uninstalling the instrumentation restores the original methods, leaving no trace in the system.

6.2. Generality beyond music

This instrumentation targets all expressions evaluated in the Playground, not any Coypu-specific code. As a result, any expression evaluated in a Phizura Playground is captured⁷, making the approach entirely generic. This opens several concrete uses beyond music. In the domain of visual live coding, performances using Bloc or Roassal for real-time graphics could be recorded and replayed with the same tool, without any modification to Phizura. Beyond live performance, Phizura can serve as a general-purpose prototyping recorder. Developers routinely explore APIs, test hypotheses, and prototype solutions through sequences of small evaluations. Phizura can capture these sessions as timestamped scripts, providing a lightweight form of session recording that preserves not just what was tried, but in what order and at what pace. Think documenting design decisions, reviewing your own process, or sharing a discovery workflow with other users.

A third application is pedagogical. An instructor demonstrating a concept in a Pharo Playground could record the session and distribute it as a small text file. Students would then replay the demonstration at their own pace, seeing each evaluation appear with the original timing. Unlike a video recording, the replayed script is live code: students can pause it, modify the expressions, and explore variations. Unlike a video recording, the replayed script is live code: students can pause it, modify the expressions, and explore variations. Unlike a static code listing, it preserves the temporal dimension, the order and rhythm of the instructor’s thought process.

6.3. Limitations

The current implementation captures Playground evaluations but does not record workspace edits that are not evaluated. A performer may type, delete, and rearrange code extensively before pressing

⁷Whether the code produces sound, generates visuals, manipulates data, or does anything else entirely.

`Cmd+D`; only the final evaluated expression is recorded. The keystroke capture partially addresses this gap by logging individual key events with timestamps and cursor positions, but it does not reconstruct the full editing history in a way that could be replayed as a visual typing sequence. That would require a different approach, closer to what Live Writing[?] provides.

Timing fidelity depends on the granularity of `DateAndTime` now and the accuracy of `seconds wait` during replay. For musical purposes, where consecutive evaluations are typically separated by one to several seconds, this is more than sufficient: the human performer’s own timing variability is far larger than any clock imprecision.

Finally, the current design supports a single recording session at a time, managed through a global variable. Collaborative live coding scenarios, where multiple performers evaluate code in separate Playgrounds, would require extending the architecture to support multiple concurrent recorders.

7. Conclusion

This paper presented Phizura, a non-intrusive recorder for live coding performances in Pharo. Using Method Proxies, the instrumentation of the Playground’s evaluation methods happens without modifying the Coypu codebase. The recorded timestamped expressions can be emitted as executable Pharo code that replays the performance with faithful timing. Compared to the previous Announcer-based implementation, the Method Proxies approach is modular, decoupled, and extensible.

Phizura demonstrates that Method Proxies, originally designed for profiling and debugging, are equally effective for creative applications. To the best of our knowledge, Phizura is the first tool to use reflective instrumentation for recording live coding performances, and the first to produce self-contained, evaluation-level replay scripts.

Future work will focus on two directions. First, we plan to expose a simpler public API within the Phizura package, allowing users to define custom handlers triggered by specific methods in their own libraries, giving live coders finer control over what is captured and how it is processed. Second, we aim to support recording across multiple Playgrounds simultaneously, enabling collaborative live coding sessions to be captured as a single, synchronized script.